

Versione originale concepita per il corso di *Gestione dei Progetti Software*- Prof.ssa F. Ferrucci

Statement of Work
Progetto
AmplyMus

Riferimento	NC07_SOW
Versione	0.1
Data	07/10/2026
Destinatario	Studenti di Ingegneria del Software 2026/27
Presentato da	Alessandro Elia Raffaele Del Vacchio Andrea Del Pezzo
Approvato da	

Revision History

Data	Versione	Descrizione	Autori
07/10/2026	0.1	Prima stesura	Alessandro Elia Raffaele Del Vacchio Andrea Del Pezzo

Statement of Work (SOW) del Progetto AmplyMus

1. Scopo del Sistema

La piattaforma intende offrire all'utenza uno spazio in cui è possibile condividere le proprie opinioni personali riguardo brani, album e artisti. Lo scopo è quello di aiutare le persone a condividere la propria passione e permettergli di ampliare i loro interessi musicali.

L'obiettivo del progetto è fornire le seguenti funzionalità:

- la fase di registrazione, autenticazione degli utenti e degli amministratori,
- la gestione del proprio account personale,
- la ricerca di artisti, album e brani musicali,
- la consultazione dei profili di altri utenti e visualizzazione delle loro attività,
- la consultazione delle schede di artisti e album (copertina, discografia, generi),
- la scrittura, la modifica e la pubblicazione di recensioni con valutazione,
- la moderazione della piattaforma per gli amministratori.

2. Data di Inizio e di Fine

Inizio: Ottobre 2026

Fine: Gennaio 2027

3. Deliverables

RAD, SDD, cenni su ODD, Matrice di Tracciabilità, Test Plan, Test Case Specification, Test incident Report, Test Summary Report, e ogni altro documento richiesto per lo sviluppo del sistema.

4. Vincoli/Constraints

Vincoli collaborativi e comunicativi.

- Rispetto scadenze delle scadenze intermedie/di fine progetto **definite nello statement of work**
- Uso di sistemi di versioning - GitHub in particolare
- Utilizzo di tool di per la suddivisione dei task e attività (Trello o similare)
- Utilizzo di tool di comunicazione tracciabile (Teams, Discord)

Vincoli tecnici

Analisi e specifica dei requisiti

- Specifica dei requisiti funzionali e non funzionali
- Specifica di minimo 2 e massimo 4 scenari per ogni membro del team;
- Diagramma degli use case
- Esattamente uno use case per ogni membro del team - i casi d'uso aggiuntivi non saranno valutati
- Specifica degli oggetti boundary, control e entity per gli use case specificati.
- Un sequence diagram ogni due membri del team - il sequence diagram deve far riferimento agli use case specificati.
- Specifica di un class diagram per team.
- Almeno un activity diagram per sistema esistente o sistema proposto e almeno uno statechart diagram ogni due membri del team.

System Design

- Specifica dei design goal e analisi dei trade-off relativi ad almeno due coppie di design goal.
- Definizione dell'architettura del sistema, Strategie relative alla gestione dei dati persistenti, Il flusso di controllo globale, Le politiche di controllo degli access

Object Design

- (Solo) Individuazione di dove poter utilizzare due design pattern, indicandone l'obiettivo e come sarebbero implementati.

Altro vincolo

• Uso di UML;

Testing

• Ogni membro dovrà effettuare il testing di unità, tramite category partition, dei casi d'uso specificati.

5. Criteri di Accettazione/Acceptance Criteria - Criteri che, se non rispettati, portano al fallimento del progetto

• Utilizzo appropriato di GitHub, che preveda il rispetto delle linee guida definite nel contesto del lab.

• Adeguato utilizzo del pull-based development, che preveda il rispetto delle linee guida definite nel contesto del lab.

• Adeguato utilizzo di un tool di comunicazione, che preveda il rispetto delle linee guida definite nel contesto del lab.

• Adeguato utilizzo di un tool per la gestione dei task, che preveda il rispetto delle linee guida definite nel contesto del lab.

• Documentazione adeguata. Verranno usati tool di plagiarism detection per identificare casi in cui gli studenti hanno copiato da progetti di anni precedenti e/o da altre fonti.

• Appropriato testing dei casi d'uso.

6. Criteri di premialità

• Uso adeguato di sistemi di build;

• Uso adeguato di un processo di continuous integration tramite Travis;

• Adozione di processi di code review;

• Uso adeguato di tool avanzati di testing (e.g., Mockito, Cobertura, etc.).